

Archivi e Basi di Dati

Dai file alle basi di dati relazionali

Informatica — Classe Quinta, Istituto Tecnico

Indice degli argomenti

1. Archivi, file e memorie di massa
2. Organizzazione degli archivi e applicazioni gestionali
3. Limiti dell'organizzazione convenzionale (file-based)
4. Basi di dati: definizione e produttività
5. Il DBMS e le sue funzioni: integrità, consistenza, sicurezza
6. Modelli dei dati per livelli di generalità: concettuale, logico, fisico
7. Il modello relazionale: approfondimento e approccio dichiarativo
8. Architettura a 3 livelli e indipendenza dei dati
9. Database distribuiti
10. Linguaggi per database: DDL, DML, QL, DCL, TCL
11. Utenti, transazioni e proprietà ACID

Gli archivi (file)

Un **archivio** (o *file*) è un insieme organizzato di **dati omogenei**, correlati tra loro, memorizzati su una **memoria di massa**.

- È composto da **record** (unità logiche), ciascuno formato da **campi** (field)
- Ogni record rappresenta un'**entità** del mondo reale (es. un cliente, uno studente)

Esempio: archivio `Studenti` → record = singolo studente → campi: `Matricola` , `Cognome` , `Nome` , `DataNascita`

File e memorie di massa

Le **memorie di massa** (hard disk, SSD) memorizzano dati in modo **permanente** (non volatile), a differenza della RAM.

Caratteristiche chiave:

- Accesso **sequenziale** o **diretto** (random)
- Organizzazione in **blocchi/settori**
- I file sono letti/scritti tramite il **file system** del sistema operativo

Memoria	Volatilità	Velocità	Capacità
RAM	Volatile	Alta	Limitata
Memoria di massa	Permanente	Minore	Elevata

Organizzazione degli archivi

Tre modalità principali di organizzazione fisica:

- **Sequenziale**: i record sono letti in ordine, uno dopo l'altro (efficiente per elaborazioni batch)
- **Sequenziale con indice (indexed-sequential)**: un indice consente di localizzare rapidamente un record, mantenendo l'ordinamento fisico
- **Ad accesso diretto (random/hashed)**: la posizione del record si calcola tramite una **funzione di hash** sulla chiave

Esempio d'esame: confrontare i tempi di ricerca su archivio sequenziale ($O(n)$) vs indicizzato ($O(\log n)$) per un archivio anagrafico di 10.000 record.

Le applicazioni gestionali

Software che automatizzano i **processi amministrativi e organizzativi** di un'azienda, basandosi su archivi di dati.

Esempi tipici:

- Gestione **magazzino** (carico/scarico merci)
- Gestione **paghe e stipendi**
- Gestione **fatturazione** e contabilità
- Gestione **prenotazioni** (alberghiere, sanitarie)

Nel modello tradizionale, ogni applicazione gestiva **i propri archivi privati**, separati da quelli delle altre applicazioni: è il modello **file-based**.

Limiti dell'organizzazione convenzionale (file-based)

Ogni programma applicativo ha i propri file dedicati. Problemi principali:

- **Ridondanza dei dati**: stesso dato duplicato in più archivi (es. indirizzo cliente ripetuto in `Ordini` e `Fatture`)
- **Inconsistenza**: aggiornando una copia e non le altre, i dati non coincidono più
- **Dipendenza dati-programmi**: modificare la struttura di un file richiede riscrivere i programmi che lo usano
- **Scarsa condivisione** dei dati tra applicazioni diverse
- **Sicurezza e integrità** poco controllate, gestite singolarmente da ogni programma
- **Difficoltà di accesso concorrente**: più utenti che modificano lo stesso file contemporaneamente rischiano di sovrascrivere le modifiche altrui

Dagli archivi indipendenti al database: il parallelismo

Problema (file-based)	Soluzione (database)
Ridondanza tra archivi separati	Dati centralizzati , ridondanza minimizzata
Inconsistenza tra copie	Unica fonte di verità , aggiornamento singolo
Dipendenza dati-programmi	Indipendenza dei dati (architettura a livelli)
Accesso concorrente non gestito	Controllo di concorrenza integrato nel DBMS
Sicurezza per singolo file	Sicurezza centralizzata (utenti, privilegi)
Nessun controllo di integrità automatico	Vincoli di integrità definiti a livello di schema

Il passaggio da **archivi indipendenti** a **basi di dati** nasce proprio per superare questi limiti strutturali, non è solo un cambio tecnologico ma organizzativo.

Basi di dati: definizione

Una **base di dati (database)** è una collezione di dati **integrata, condivisa e persistente**, gestita da un software chiamato **DBMS** (Data Base Management System).

Principi cardine:

- **Centralizzazione** dei dati (unica fonte)
- **Condivisione** tra più applicazioni e utenti
- **Controllo della ridondanza** (non sempre eliminata del tutto, ma gestita e nota al sistema)
- **Indipendenza** tra dati e programmi

Il DBMS è l'interfaccia tra gli **utenti/applicazioni** e i dati fisicamente memorizzati: nessuna applicazione accede ai file "grezzi" direttamente.

Basi di dati e produttività

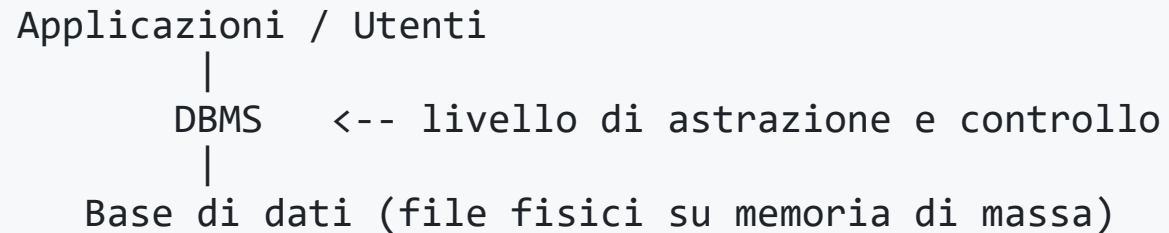
L'adozione di un database migliora la **produttività** a più livelli:

- **Sviluppo più rapido delle applicazioni**: i programmatori non devono gestire manualmente la struttura fisica dei dati
- **Riuso dei dati**: la stessa base di dati serve più applicazioni (es. magazzino, vendite, contabilità)
- **Riduzione degli errori**: i vincoli di integrità sono verificati automaticamente dal DBMS, non dal singolo programma
- **Tempestività delle informazioni**: query e report immediati, senza dover scrivere nuovi programmi ad hoc
- **Supporto alle decisioni**: interrogazioni complesse (query, aggregazioni) forniscono informazioni utili al management in tempi rapidi

Esempio pratico: un'azienda che passa da fogli Excel separati a un database centralizzato riduce drasticamente il tempo per produrre un report vendite mensile.

Il DBMS: cos'è

Il **DBMS (Data Base Management System)** è il software che si interpone tra gli utenti/applicazioni e i dati fisici, offrendo un insieme di servizi per **definire, manipolare, interrogare e proteggere** i dati.



Non si accede mai direttamente ai file: ogni operazione passa attraverso il DBMS, che garantisce coerenza, sicurezza e prestazioni.

Le funzioni del DBMS (1/2)

- **Definizione dei dati**: permette di descrivere schema, tipi, vincoli tramite il **DDL**
- **Manipolazione dei dati**: inserimento, modifica, cancellazione, lettura tramite **DML**
- **Interrogazione dei dati**: estrazione di informazioni tramite un **QL** (Query Language)
- **Controllo della concorrenza**: gestisce l'accesso simultaneo di più utenti evitando conflitti (locking, isolamento)
- **Gestione della sicurezza**: autenticazione degli utenti e autorizzazione tramite **privilegi/permessi**
(GRANT / REVOKE)

Le funzioni del DBMS (2/2)

- **Garanzia dell'integrità**: verifica automatica dei vincoli (dominio, chiave, referenziale, regole applicative)
- **Affidabilità (backup e recovery)**: attraverso **log delle transazioni** e **checkpoint**, consente di ripristinare il database dopo un guasto
- **Dizionario dei dati (catalogo di sistema)**: metadati su tabelle, attributi, vincoli, utenti — è "il database che descrive il database"
- **Efficienza**: un **ottimizzatore di query** sceglie il piano di esecuzione più efficiente, sfruttando **indici** sulle tabelle

Domanda tipo esame: "Elenca e descrivi almeno quattro funzioni fondamentali di un DBMS, facendo un esempio per ciascuna."

Integrità dei dati

L'**integrità** garantisce che i dati memorizzati siano **corretti, validi e coerenti con le regole del dominio applicativo**. Il DBMS la controlla tramite **vincoli (constraint)**:

- **Vincolo di dominio**: un attributo può assumere solo valori appartenenti a un insieme definito (es. `Voto` tra 0 e 10)
- **Vincolo di chiave (integrità di entità)**: la chiave primaria non può essere nulla né duplicata
- **Vincolo di integrità referenziale**: una chiave esterna deve corrispondere a un valore esistente nella tabella referenziata
- **Vincoli utente/di business**: regole specifiche dell'applicazione (es. `DataConsegna >= DataOrdine`)

Esempio: `CHECK (Voto BETWEEN 0 AND 10)` in una tabella `Esami` impedisce l'inserimento di voti non validi.

Consistenza dei dati

La **consistenza** è la proprietà per cui il database si trova sempre in uno **stato valido**, coerente con tutti i vincoli di integrità definiti, **prima e dopo** ogni operazione.

- È strettamente legata all'integrità: rispettare i vincoli **garantisce** la consistenza
- Diventa cruciale nelle **transazioni** e nell'**accesso concorrente**: più utenti che operano insieme non devono portare il database in uno stato incoerente

Esempio: se un vincolo referenziale collega `Ordini.CodCliente` a `Clienti.CodCliente`, il DBMS impedisce di cancellare un cliente che ha ancora ordini attivi (o applica un'azione `ON DELETE CASCADE / RESTRICT`), mantenendo così la consistenza.

I modelli dei dati: livelli di generalità

Un **modello dei dati** è il formalismo con cui si rappresenta la realtà da memorizzare. Si distinguono **tre livelli**, coerenti con le fasi di progettazione già viste, che vanno da un livello più **astratto e generale** a uno più **concreto e specifico**:

Livello	Cosa rappresenta	Unità di base
Modello concettuale	La realtà in modo astratto, indipendente da ogni tecnologia	oggetto (entità)
Modello logico	La struttura dei dati secondo un formalismo implementabile in un DBMS	record
Modello fisico	Come i dati sono effettivamente memorizzati su memoria di massa	file, blocchi, indici

Man mano che si scende di livello, il modello diventa più **specifico** e più legato alla tecnologia: dal "cosa" (concettuale) al "come" (fisico).

Il modello concettuale: livello di oggetti

Al **livello concettuale** la realtà viene rappresentata attraverso **oggetti** (entità) dotati di proprietà e collegati da legami logici, **senza alcun riferimento** a come i dati verranno poi implementati.

- Lo strumento standard per costruire il modello concettuale è il **modello Entità/Associazioni (E/R)**, già approfondito nella progettazione concettuale: entità, associazioni, attributi, cardinalità
- Il risultato è lo **schema concettuale** (diagramma ER), indipendente dal DBMS che verrà scelto in seguito

A questo livello ci si chiede *"quali oggetti esistono nella realtà e come sono collegati?"*, non ancora *"come li memorizzo?"*.

Il modello logico: livello di record

Al **livello logico** i dati vengono organizzati secondo un **formalismo implementabile** da un DBMS: l'unità di base non è più l'oggetto astratto, ma il **record**, cioè la struttura concreta con cui il dato viene rappresentato e manipolato.

Il modello logico può essere realizzato secondo diversi **modelli dei dati**:

- **Modello gerarchico**: record organizzati ad **albero** (relazioni 1:N)
- **Modello reticolare (a rete)**: generalizza il gerarchico, ammette relazioni N:N tramite puntatori tra record
- **Modello relazionale**: record organizzati in **tabelle (relazioni)**, oggi il più diffuso
- **Modello a oggetti**: dati rappresentati come **oggetti** con attributi e metodi (usato in contesti OOP)

Il passaggio dal modello concettuale (E/R) al modello logico si chiama **progettazione logica**: ad esempio, ogni entità del diagramma ER diventa una tabella nel modello relazionale.

Il modello fisico

Al **livello fisico** si definisce **come** i record vengono effettivamente memorizzati sulla memoria di massa: organizzazione dei file (sequenziale, a indice, hash), tipi di indice, allocazione dei blocchi.

- È il livello più vicino alla macchina e dipende dal **DBMS specifico** utilizzato
- Le scelte fisiche (es. creare un indice su una colonna) **non modificano** lo schema logico, grazie all'**indipendenza fisica dei dati** già vista nell'architettura a 3 livelli

Esempio: decidere di indicizzare la colonna `Cognome` per velocizzare le ricerche è una scelta di livello **fisico**, trasparente per chi scrive query SQL a livello logico.

Il modello gerarchico e reticolare (cenni storici)

Prima dell'affermazione del modello relazionale, i DBMS si basavano su altri modelli logici, oggi in gran parte superati ma utili per comprenderne l'evoluzione:

- **Modello gerarchico**: i record sono organizzati come un **albero**; ogni record "figlio" ha un solo record "padre" (relazioni **1:N**). Limite: difficoltà a rappresentare relazioni **N:N** in modo naturale
- **Modello reticolare (CODASYL)**: generalizza il gerarchico introducendo **puntatori** tra record che permettono relazioni **N:N**, ma con **navigazione esplicita** tra i record da parte del programmatore

Entrambi i modelli richiedono che il programmatore **conosca il percorso di accesso** ai dati (navigazione tra puntatori): è un approccio **procedurale**, come vedremo tra poco a confronto col modello relazionale.

Il modello relazionale: concetti base

- **Relazione** = tabella; **tupla** = riga (record); **attributo** = colonna (campo)
- **Dominio**: insieme dei valori ammissibili per un attributo (es. dominio di `voto` = interi 0–10)
- **Grado** della relazione: numero di attributi (colonne)
- **Cardinalità** della relazione: numero di tuple (righe)
- Non esiste un ordinamento significativo tra righe né tra colonne: l'accesso avviene per **valore**, non per posizione

Esempio: `Studenti(Matricola, Cognome, Nome, Classe)` → grado 4, cardinalità = numero di studenti.

Il modello relazionale: le chiavi

- **Superchiave**: insieme di attributi che identifica univocamente una tupla (può contenere attributi ridondanti)
- **Chiave candidata**: superchiave minimale (nessun attributo è superfluo)
- **Chiave primaria (PK)**: la chiave candidata scelta per identificare univocamente le tuple; non può essere NULL
- **Chiave esterna (FK)**: attributo (o gruppo) che fa riferimento alla chiave primaria di un'altra tabella, per realizzare le **associazioni** tra entità

Esempio: Esami(Matricola FK, Materia, Voto) → Matricola è chiave esterna verso Studenti(Matricola PK) .

Modello relazionale: approccio dichiarativo vs procedurale

Il vero salto di qualità del modello relazionale rispetto a gerarchico e reticolare è il passaggio da un approccio **procedurale** a uno **dichiarativo**:

	Approccio procedurale (gerarchico/reticolare)	Approccio dichiarativo (relazionale)
Cosa specifica il programmatore	Come ottenere il dato (percorso di accesso passo-passo, navigazione tra puntatori)	Cosa si vuole ottenere (il risultato desiderato)
Chi decide il percorso di accesso	Il programmatore	Il DBMS , tramite l'ottimizzatore di query
Esempio di linguaggio	Istruzioni di navigazione record-per-record	<code>SELECT</code> in SQL

Esempio pratico: per ottenere "tutti gli studenti con media voto ≥ 8 ", in un modello procedurale il programmatore dovrebbe scrivere un ciclo che scorre manualmente i record; in SQL basta dichiarare **cosa** si vuole:

```
SELECT Cognome FROM Studenti JOIN Esami ... GROUP BY ... HAVING AVG(Voto) >= 8;
```

Questa caratteristica rende il modello relazionale più semplice da usare e **indipendente dai dettagli di accesso fisico** ai dati.

Vincoli di integrità relazionale

- **Integrità di entità**: la chiave primaria di ogni tupla deve essere **non nulla** e **unica**
- **Integrità referenziale**: ogni valore di chiave esterna deve corrispondere a un valore esistente della chiave primaria referenziata (oppure essere `NULL`, se ammesso)

```
CREATE TABLE Esami (  
  Matricola INT,  
  Materia VARCHAR(30),  
  Voto INT CHECK (Voto BETWEEN 0 AND 10),  
  FOREIGN KEY (Matricola) REFERENCES Studenti(Matricola)  
);
```

Questi vincoli sono il fondamento della **consistenza** vista in precedenza: il DBMS li verifica automaticamente ad ogni operazione.

Algebra relazionale: operatori principali

L'**algebra relazionale** è il fondamento teorico delle interrogazioni SQL. Operatori principali:

Operatore	Significato	Equivalente SQL
Selezione (σ)	Filtra le righe secondo una condizione	WHERE
Proiezione (π)	Seleziona un sottoinsieme di colonne	SELECT col1, col2
Congiunzione ($\bowtie$, Join)	Combina righe di tabelle diverse secondo una condizione	JOIN ... ON
Unione ($\cup$)	Unisce i risultati di due interrogazioni compatibili	UNION
Differenza ($-$)	Righe presenti in una relazione ma non nell'altra	EXCEPT (MySQL 8.0.31+)

Esempio: $\sigma(\text{Voto} \geq 8)(\text{Esami}) \bowtie \text{Studenti} \rightarrow$ equivalente a `SELECT * FROM Esami JOIN Studenti ON ... WHERE Voto >= 8;`

Cenni di normalizzazione

La **normalizzazione** è il processo che elimina ridondanze e anomalie organizzando gli attributi in tabelle ben progettate, tramite le **forme normali**:

- **1FN (1NF)**: ogni attributo contiene un valore **atomico** (non ripetuto, non composito)
- **2FN (2NF)**: rispetta la 1FN e ogni attributo non chiave dipende dall'**intera** chiave primaria (non da una sua parte)
- **3FN (3NF)**: rispetta la 2FN e non esistono **dipendenze transitive** tra attributi non chiave

Esempio: se in `Ordini(NumOrdine, CodCliente, NomeCliente)` il `NomeCliente` dipende da `CodCliente` e non dalla chiave `NumOrdine`, si ha una dipendenza transitiva da eliminare separando `Clienti` in una tabella a parte.

Architettura a 3 livelli (ANSI-SPARC)

Il DBMS è organizzato secondo tre livelli di astrazione, per garantire **indipendenza dei dati**:

- **Livello esterno**: le **viste** personalizzate per i singoli utenti/applicazioni
- **Livello concettuale (logico)**: la struttura complessiva dei dati (schema logico, es. tabelle e vincoli)
- **Livello interno (fisico)**: come i dati sono effettivamente memorizzati su disco (file, indici)



Indipendenza dei dati

L'architettura a 3 livelli garantisce due tipi di **indipendenza**:

- **Indipendenza fisica**: si può modificare l'organizzazione fisica dei dati (es. cambiare tipo di indice) **senza** modificare lo schema logico né i programmi applicativi
- **Indipendenza logica**: si può modificare lo schema concettuale (es. aggiungere una tabella) **senza** impattare le viste esistenti degli utenti

Domanda tipo esame: "Spiega perché l'indipendenza dei dati è un vantaggio rispetto all'organizzazione file-based"
→ riduce l'impatto delle modifiche, aumenta la manutenibilità.

Database distribuiti: definizione

Un **database distribuito (DDB)** è un insieme di basi di dati **logicamente correlate** ma **fisicamente dislocate** su più nodi (server), collegati da una rete, che appaiono all'utente come un **unico sistema**.

- Gestito da un **DDBMS** (Distributed DBMS)
- Obiettivo: **trasparenza** per l'utente (non deve sapere dove sono fisicamente i dati)

Tecniche fondamentali:

- **Frammentazione orizzontale**: righe di una tabella distribuite su più nodi (es. clienti Nord/Sud su server diversi)
- **Frammentazione verticale**: colonne di una tabella distribuite su più nodi
- **Replicazione**: copie degli stessi dati su più nodi, per disponibilità e prestazioni

Database distribuiti: vantaggi e svantaggi

Vantaggi	Svantaggi
Maggiore disponibilità (un nodo guasto non blocca tutto il sistema)	Complessità di progettazione e gestione
Scalabilità orizzontale (si aggiungono nodi)	Difficoltà nel mantenere la consistenza tra nodi
Dati vicini agli utenti che li usano (minore latenza)	Overhead di comunicazione di rete
Tolleranza ai guasti (fault tolerance)	Transazioni distribuite più complesse (commit a due fasi)

Cenno teorico (Teorema di CAP): un sistema distribuito non può garantire simultaneamente **Consistenza**, **Disponibilità** e **Tolleranza di partizione**; bisogna scegliere quali privilegiare in base al contesto.

Linguaggi per database: panoramica

Il linguaggio standard per i database relazionali è **SQL (Structured Query Language)**, suddiviso in sottolinguaggi con scopi distinti:

Sottolinguaggio	Scopo	Comandi principali
DDL	Definizione della struttura	CREATE , ALTER , DROP
DML	Manipolazione dei dati	INSERT , UPDATE , DELETE
QL / DQL	Interrogazione dei dati	SELECT
DCL	Controllo degli accessi	GRANT , REVOKE
TCL	Controllo delle transazioni	COMMIT , ROLLBACK

DDL — Data Definition Language

Definisce e modifica la **struttura** del database: tabelle, vincoli, indici.

```
CREATE TABLE Studenti (  
    Matricola INT AUTO_INCREMENT PRIMARY KEY,  
    Cognome VARCHAR(30) NOT NULL,  
    Nome VARCHAR(30) NOT NULL,  
    Classe VARCHAR(5)  
) ENGINE=InnoDB;  
  
ALTER TABLE Studenti ADD Email VARCHAR(50);  
  
DROP TABLE Studenti;
```

Da ricordare per l'esame: CREATE definisce, ALTER modifica la struttura esistente, DROP elimina l'oggetto (tabella, vincolo, indice). In MySQL, AUTO_INCREMENT genera automaticamente un valore progressivo per la chiave primaria e ENGINE=InnoDB è il motore di archiviazione predefinito (supporta chiavi esterne e transazioni).

DML — Data Manipulation Language

Permette di **manipolare il contenuto** delle tabelle: inserire, aggiornare, eliminare tuple.

```
INSERT INTO Studenti (Matricola, Cognome, Nome, Classe)
VALUES (1001, 'Rossi', 'Anna', '5A');

UPDATE Studenti SET Classe = '5B' WHERE Matricola = 1001;

DELETE FROM Studenti WHERE Matricola = 1001;
```

Ogni operazione DML può violare un vincolo di integrità: in tal caso il DBMS **rifiuta l'operazione** e segnala l'errore.

QL — Query Language (interrogazioni)

Il comando `SELECT` costituisce il **linguaggio di interrogazione (QL)**, corrispondente all'algebra relazionale vista in precedenza.

```
-- Selezione + proiezione
SELECT Cognome, Nome FROM Studenti WHERE Classe = '5A';

-- Join tra tabelle
SELECT S.Cognome, E.Materia, E.Voto
FROM Studenti S
JOIN Esami E ON S.Matricola = E.Matricola
WHERE E.Voto >= 8;

-- Aggregazione (tipica delle seconde prove)
SELECT Classe, AVG(Voto) AS MediaVoti
FROM Studenti S JOIN Esami E ON S.Matricola = E.Matricola
GROUP BY Classe
HAVING AVG(Voto) >= 7;
```

DCL e TCL

DCL (Data Control Language) — gestisce i **privilegi** di accesso:

```
GRANT SELECT, INSERT ON scuola.Studenti TO 'docente'@'localhost';  
REVOKE INSERT ON scuola.Studenti FROM 'docente'@'localhost';
```

TCL (Transaction Control Language) — gestisce l'esito delle **transazioni**:

```
START TRANSACTION;  
UPDATE Conti SET Saldo = Saldo - 100 WHERE IdConto = 1;  
UPDATE Conti SET Saldo = Saldo + 100 WHERE IdConto = 2;  
COMMIT;    -- oppure ROLLBACK; in caso di errore
```

In MySQL i privilegi si assegnano a un **utente identificato da nome e host** ('utente'@'host'); `START TRANSACTION` avvia esplicitamente una transazione (in alternativa è disponibile anche `BEGIN;`).

Utenti di un database

Diverse figure interagiscono con il sistema, con ruoli e permessi differenti:

- **DBA (Database Administrator)**: progetta lo schema, amministra sicurezza, prestazioni, backup e recovery
- **Progettisti e programmatori (analisti/sviluppatori)**: progettano e sviluppano le applicazioni che accedono al DB
- **Utenti finali**:
 - *Occasional*: eseguono query non ripetitive, spesso in SQL diretto
 - *Abituali/parametrici*: eseguono operazioni predefinite tramite maschere applicative (form, report)

Ogni categoria di utente ha **privilegi differenziati**, gestiti tramite DCL (GRANT / REVOKE).

Le transazioni

Una **transazione** è una **sequenza di operazioni** sul database, eseguita come **unità logica indivisibile**.

- Porta il database da uno **stato consistente** a un altro **stato consistente**
- Termina con **COMMIT** (conferma definitiva) oppure **ROLLBACK** (annullamento)

Esempio classico: trasferimento di denaro tra due conti bancari →

1. addebito conto A, 2) accredito conto B.

Se una delle due operazioni fallisce, l'intera transazione va annullata (rollback), per non violare la consistenza.

Le proprietà ACID

Ogni transazione deve rispettare le proprietà **ACID**:

- **Atomicità (Atomicity)**: la transazione è **tutto o niente** — se un'operazione fallisce, si annullano tutte
- **Coerenza (Consistency)**: la transazione porta il DB da uno stato valido a un altro stato valido, rispettando i vincoli
- **Isolamento (Isolation)**: transazioni concorrenti non si influenzano a vicenda (come se fossero eseguite in sequenza)
- **Durabilità (Durability)**: una volta confermata (commit), la modifica è **permanente**, anche in caso di guasto del sistema

ACID: esempio riassuntivo

Scenario da simulazione d'esame: un sistema di prenotazione voli gestisce l'acquisto dell'ultimo posto disponibile da parte di due utenti contemporaneamente.

- **Atomicità:** l'operazione di prenotazione (verifica posto + decremento disponibilità) è indivisibile
- **Isolamento:** i due utenti non devono vedere risultati intermedi l'uno dell'altro
- **Coerenza:** il numero di posti non può mai diventare negativo
- **Durabilità:** una volta confermata la prenotazione, resta salvata anche in caso di crash del server

Mapa concettuale riassuntiva

```
ARCHIVI (file-based)
|-- Limiti: ridondanza, inconsistenza, dipendenza dati-programmi
V
BASI DI DATI (DBMS) -- produttivita', condivisione, sicurezza
|-- Funzioni DBMS: DDL/DML/QL, integrita', concorrenza,
|                   sicurezza, backup&recovery, dizionario dati
|-- Livelli di generalita': concettuale (oggetti, E/R) --
|                   logico (record: gerarchico/reticolare/relazionale) --
|                   fisico (file, indici)
|-- Modello relazionale: tabelle, chiavi, vincoli, algebra relazionale,
|                   approccio dichiarativo (vs procedurale)
|-- Architettura a 3 livelli --> Indipendenza dei dati (fisica/logica)
|-- Database distribuiti: frammentazione, replicazione, trasparenza
|-- Linguaggi: DDL, DML, QL, DCL, TCL (SQL)
|-- Utenti: DBA, programmatori, utenti finali
|-- Transazioni --> proprieta' ACID
```

Domande guida per il ripasso

- Perché il modello file-based genera ridondanza e inconsistenza? Come lo risolve un database?
- Elenca le funzioni principali di un DBMS con un esempio per ciascuna
- Qual è la differenza tra integrità e consistenza dei dati?
- Descrivi i tre livelli di generalità di un modello dei dati (concettuale, logico, fisico) e cosa rappresenta l'unità di base di ciascuno
- Spiega la differenza tra approccio procedurale e dichiarativo, con un esempio riferito al modello relazionale
- Descrivi chiavi primarie, candidate ed esterne nel modello relazionale
- Quali sono i tre livelli dell'architettura ANSI-SPARC e a cosa serve ciascuno?
- Cos'è un database distribuito e quali sono i suoi vantaggi/svantaggi?
- Scrivi query SQL con DDL, DML e QL su un caso a scelta
- Spiega le 4 proprietà ACID con un esempio concreto

Fine

Domande?